



# UNIVERSIDAD DE COSTA RICA

## Sede del Atlántico

# RP Recinto de Paraíso

UNIVERSIDAD DE COSTA RICA  
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO  
CARRERA DE INFORMÁTICA EMPRESARIAL  
CURSO: IF0009 - Desarrollo de Software IV  
PROFESOR: Mag. Jonathan Granados C.  
SEMESTRE: II-2026

---

## Taller 1: Arquitectura Cliente-Servidor y Maquetación Web Semántica (HTML, CSS y Java)

---

## Metadatos

---

- **Tiempo Estimado:** 3 horas
- **Herramientas Requeridas:**
  - JDK 17 o superior instalado y configurado en el PATH del sistema.
  - VS Code u otro editor de código de su elección.
  - Navegador web moderno (Google Chrome, Mozilla Firefox o Microsoft Edge).
- **Metas de Aprendizaje:**
  - Construir un sitio web responsivo estructurado con HTML5 semántico y estilizado con CSS3 moderno utilizando Flexbox y Variables CSS.
  - Implementar y ejecutar un servidor HTTP nativo en Java para entender el flujo de petición (Request) y respuesta (Response) en la arquitectura cliente-servidor.
  - Utilizar las herramientas de desarrollo del navegador (DevTools) para inspeccionar tráfico de red, modificar hojas de estilo y depurar problemas comunes.

---

## Introducción (Lectura y Conceptos)

En el desarrollo de software moderno, la **arquitectura cliente-servidor** es el pilar fundamental de la Web: \* **El Cliente (Front-end):** Reside en el navegador del usuario. Es el responsable de estructurar la información (HTML), aplicar la estética y el diseño visual (CSS) e implementar el dinamismo interactivo (JavaScript). \* **El Servidor (Back-end):** Se ejecuta en una máquina remota. Escucha activamente peticiones en un puerto específico, procesa las solicitudes, interactúa con la lógica de negocio y bases de datos, y devuelve recursos (HTML, CSS, JSON, etc.) al cliente.

En este taller de nivel inicial, usted creará un cliente web básico y un servidor HTTP en Java desde cero para ver de forma práctica cómo interactúan estas dos capas en tiempo real.

---

## Parte 1: Práctica Guiada (Paso a Paso)

---

## Paso 1: Estructuración del Cliente Web (HTML5)

Cree un archivo llamado `index.html` en una carpeta de su elección. Este archivo de texto plano estructurará la información utilizando etiquetas semánticas de HTML5.

[!NOTE] Las etiquetas semánticas como `<header>`, `<main>`, `<section>` y `<footer>` no tienen un comportamiento visual especial por sí mismas, pero le indican al navegador y a los motores de búsqueda el propósito exacto de cada sección del documento.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Perfil Estudiantil - UCR</title>
  <!-- Vinculación de la hoja de estilos externa -->
  <link rel="stylesheet" href="/styles.css">
</head>
<body>
  <header class="app-header">
    <h1>Portal Universitario - UCR</h1>
    <p>Curso: IF0009 - Desarrollo de Software IV</p>
  </header>
  <main class="app-content">
    <section class="profile-card">
      <div class="profile-header">
        <h2>Información del Estudiante</h2>
      </div>
      <div class="profile-body">
        <p><strong>Nombre:</strong> Estudiante UCR</p>
        <p><strong>Carné:</strong> C12345</p>
        <p><strong>Sede:</strong> Sede del Atlántico, Recinto Paraíso</p>
        <p><strong>Carrera:</strong> Informática Empresarial</p>
      </div>
    </section>
  </main>
  <footer class="app-footer">
    <p>© 2026 Universidad de Costa Rica - Informática Empresarial</p>
  </footer>
</body>
</html>
```

---

## Paso 2: Diseño del Cliente Web (CSS3)

Cree un archivo llamado `styles.css` en la misma carpeta que su archivo `index.html`. Utilizaremos variables CSS para la paleta de colores y Flexbox para posicionar los elementos en pantalla.

```
/* Declaración de variables CSS en el ámbito global (:root) */
:root {
  --color-principal: #0b0f19;
  --color-acento: #38bdf8;
  --color-fondo-tarjeta: #1e293b;
  --color-texto: #f8fafc;
  --color-borde: #334155;
  --radio-borde: 8px;
}

/* Estilos globales y reset simple */
body {
  margin: 0;
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
  background-color: var(--color-principal);
  color: var(--color-texto);
  min-height: 100vh;
  display: flex;
  flex-direction: column;
}

/* Encabezado */
.app-header {
  background-color: var(--color-fondo-tarjeta);
  border-bottom: 1px solid var(--color-borde);
  padding: 20px;
  text-align: center;
}

.app-header h1 {
  margin: 0;
  color: var(--color-acento);
  font-size: 1.8rem;
}

.app-header p {
  margin: 5px 0 0 0;
  color: #94a3b8;
}
```

```

/* Área de contenido central */
.app-content {
  flex-grow: 1;
  display: flex;
  justify-content: center;
  align-items: center;
  padding: 20px;
}

/* Tarjeta de Perfil */
.profile-card {
  background-color: var(--color-fondo-tarjeta);
  border: 1px solid var(--color-borde);
  border-radius: var(--radio-borde);
  width: 100%;
  max-width: 400px;
  box-shadow: 0 10px 15px -3px rgba(0, 0, 0, 0.3);
  transition: transform 0.2s ease, box-shadow 0.2s ease;
}

/* Animación al pasar el mouse por encima */
.profile-card:hover {
  transform: translateY(-5px);
  box-shadow: 0 15px 25px -5px rgba(56, 189, 248, 0.2);
}

.profile-header {
  border-bottom: 1px solid var(--color-borde);
  padding: 15px 20px;
}

.profile-header h2 {
  margin: 0;
  font-size: 1.25rem;
  color: var(--color-acento);
}

.profile-body {
  padding: 20px;
}

.profile-body p {
  margin: 10px 0;
  font-size: 1rem;
}

```

```

.profile-body strong {
  color: var(--color-acento);
}

/* Pie de página */
.app-footer {
  background-color: var(--color-fondo-tarjeta);
  border-top: 1px solid var(--color-borde);
  padding: 15px;
  text-align: center;
  font-size: 0.85rem;
  color: #64748b;
}

```

---

### Paso 3: Construcción del Servidor HTTP (Java)

Para entender cómo funciona el servidor, crearemos un archivo Java llamado `ServidorLocal.java` en la misma carpeta. Este servidor utilizará la biblioteca nativa `com.sun.net.httpserver` incluida en el JDK para servir nuestros archivos estáticos y registrar información en la consola sobre cada petición que recibe.

**[!IMPORTANT]** Recuerde que al programar el código Java, cada línea debe estar correctamente estructurada. En sistemas de producción usted utilizará frameworks complejos como Spring Boot, pero para este taller básico usaremos las herramientas internas del JDK para entender el flujo a bajo nivel.

```

import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;
import com.sun.net.httpserver.HttpServer;
import java.io.File;
import java.io.FileInputStream;
import java.io.IOException;
import java.io.OutputStream;
import java.net.InetSocketAddress;

public class ServidorLocal {

    public static void main(String[] args) throws IOException {
        int puerto = 8080;
        // Crea el servidor en la dirección local y puerto especificado
    }
}

```

```

        HttpServer servidor = HttpServer.create(
            new InetSocketAddress(puerto), 0
        );

        // Define el contexto "/" para manejar las peticiones de archivos
        servidor.createContext("/", new ManejadorArchivos());

        System.out.println("Servidor local iniciado en el puerto " + puerto);
        System.out.println("Ingrese a: http://localhost:" + puerto);
        servidor.start();
    }

    static class ManejadorArchivos implements HttpHandler {
        @Override
        public void handle(HttpExchange intercambio) throws IOException {
            // Obtenemos la ruta solicitada por el navegador
            String rutaSolicitada = intercambio.getRequestURI().getPath();

            // Si la ruta está vacía o es la raíz, servimos index.html
            if (rutaSolicitada.equals("/") || rutaSolicitada.isEmpty()) {
                rutaSolicitada = "/index.html";
            }

            // Buscamos el archivo correspondiente en el directorio actual
            File archivo = new File(".") + rutaSolicitada;
            System.out.println("Petición recibida: [" +
                intercambio.getRequestMethod() + "] " + rutaSolicitada);

            if (archivo.exists() && !archivo.isDirectory()) {
                // Definimos el tipo MIME del archivo para el navegador
                String tipoMIME = obtenerTipoMIME(rutaSolicitada);
                intercambio.getResponseHeaders().set("Content-Type", tipoMIME);
                intercambio.sendResponseHeaders(200, archivo.length());

                // Leemos el archivo y lo enviamos al cliente
                try (OutputStream os = intercambio.getResponseBody();
                    FileInputStream fis = new FileInputStream(archivo)) {
                    byte[] buffer = new byte[1024];
                    int bytesLeidos;
                    while ((bytesLeidos = fis.read(buffer)) != -1) {
                        os.write(buffer, 0, bytesLeidos);
                    }
                }
            } else {
                // Si el archivo no existe, respondemos con código de error 404
                String mensaje404 = "<h1>404 Recurso No Encontrado</h1>";
            }
        }
    }

```

```

        intercambio.getResponseHeaders().set(
            "Content-Type", "text/html; charset=UTF-8"
        );
        intercambio.sendResponseHeaders(404, mensaje404.length());
        try (OutputStream os = intercambio.getResponseBody()) {
            os.write(mensaje404.getBytes());
        }
    }
}

private String obtenerTipoMIME(String ruta) {
    if (ruta.endsWith(".html")) return "text/html; charset=UTF-8";
    if (ruta.endsWith(".css")) return "text/css; charset=UTF-8";
    if (ruta.endsWith(".js")) return "application/javascript";
    return "text/plain";
}
}
}

```

---

## Paso 4: Ejecución y Pruebas del Sistema

1. Abra una terminal en la carpeta donde guardó los tres archivos anteriores (`index.html`, `styles.css`, `ServidorLocal.java`).
  2. Compile y ejecute el servidor Java directamente mediante el siguiente comando en la consola: `bash java ServidorLocal.java` (Nota: A partir de Java 11, usted puede ejecutar archivos de clase única directamente sin compilar previamente con `javac`).
  3. Debería ver el siguiente mensaje en la terminal: `text Servidor local iniciado en el puerto 8080` Ingrese a: `http://localhost:" + puerto`
  4. Abra su navegador web e ingrese a `http://localhost:8080`. Verá su portal de perfil universitario renderizado con los estilos correctos.
  5. Regrese a la terminal; verá el registro impreso de las peticiones HTTP que el navegador envió para cargar los archivos: `text Petición recibida: [GET] /index.html Petición recibida: [GET] /styles.css`
-



## Parte 2: Depuración (Gotchas y Diagnósticos)

---

### El Error Común: Recurso No Cargado (Status 404)

Es muy común que, al mover archivos o configurar rutas relativas, el cliente intente solicitar archivos que no existen en el servidor, o que el servidor no pueda encontrar debido a errores de código.

#### Ejercicio de Simulación del Error:

1. Modifique temporalmente el archivo `index.html` y cambie la línea de vinculación del estilo a una ruta incorrecta: `html <link rel="stylesheet" href="/estilo-inexistente.css">`
2. Recargue la página `http://localhost:8080` en su navegador.
3. Observará que el portal pierde por completo todo el diseño visual, mostrando texto sin color y fondos blancos.

#### Procedimiento de Diagnóstico con DevTools del Navegador:

- [ ] Abra las herramientas de desarrollo del navegador presionando la tecla **F12** (o haciendo clic derecho en cualquier parte de la página y seleccionando **Inspeccionar**).
- [ ] Diríjase a la pestaña **Network** (Red) y recargue la página nuevamente (F5).
- [ ] Identifique la petición fallida (aparecerá resaltada en color rojo con el nombre `estilo-inexistente.css`).
- [ ] Seleccione el archivo fallido y examine los detalles:
- **Status Code:** Debería ver `404 Not Found`. Esto confirma que el navegador realizó la petición, pero nuestro servidor Java devolvió un código indicando que no localizó el recurso.
- [ ] Corrija la línea en su archivo `index.html` restaurando el nombre correcto `/styles.css` y recargue la página para validar que el error desaparece.

---

## Parte 3: Reto Autónomo (Sin Solución Explícita)

---

Para consolidar lo aprendido, usted deberá ampliar las capacidades del cliente y del servidor para manejar datos dinámicos mediante una API REST en miniatura:

## Requerimientos del Reto:

1. **En el Servidor Java (`ServidorLocal.java`):**
2. Agregue un nuevo endpoint o condición dentro de su manejador para capturar la ruta `/api/estudiante`.
3. Si el cliente solicita `/api/estudiante`, el servidor debe responder con un texto en formato **JSON** que contenga datos académicos de un estudiante.
4. La respuesta JSON debe tener el siguiente formato exacto: `json {"nombre": "Su Nombre", "carnet": "Su Carné", "promedio": 9.5}`
5. Asegúrese de definir el encabezado `Content-Type` de esta respuesta como `application/json; charset=UTF-8` en los encabezados del HTTP exchange.
6. **En el Cliente Web (`index.html`):**
7. Agregue un botón en la tarjeta de perfil con el texto "Cargar Notas".
8. Agregue un contenedor vacío en el HTML (ej. `<div id="resultado-nota"></div>`).
9. Escriba un script de JavaScript antes de cerrar la etiqueta `</body>` que escuche el clic del botón. Al ocurrir el clic, debe realizar una petición asíncrona (`fetch("/api/estudiante")`), transformar la respuesta a JSON e insertar el promedio obtenido dinámicamente dentro del contenedor HTML sin refrescar la página.

## Verificación del Éxito:

- Al presionar el botón "Cargar Notas", el promedio del estudiante debe desplegarse en la pantalla de forma instantánea.
- En la consola del servidor Java en la terminal, debe quedar registrado el ingreso de la petición `[GET] /api/estudiante` con código de respuesta exitoso (200).
- En la pestaña Network del navegador, la petición a `/api/estudiante` debe registrar un Status 200 OK y mostrar el objeto JSON correspondiente.